

MiniRay Stage 1 · Week 01 · Day 1 TODO

工程骨架初始化: CMake + Git + 测试入口

建议用时: 4 小时 | 交付方式: 可构建、可测试、可运行、可提交

来源依据: MiniRay Stage 1 Plan 的 Week 01 / Day 1: 建立 repo、写顶层 CMakeLists.txt、约定 C++20 与 warning flags, 并完成 miniray_hello + smoke test。

今日总目标 1 个空壳工程	建议时长 4 h	源文件/文档 6+ 个	验收命令 4 条全过	最终动作 1 次 Git commit
------------------	-------------	----------------	---------------	------------------------

1. 今日唯一目标

在 4 小时内, 把 MiniRay 初始化成一个“能一键构建、一键测试、能运行 hello demo”的 C++20 工程骨架。

边界控制: 今天不做 Ray 源码阅读、不做线程池、不接 gRPC、不写 ObjectStore、不设计调度器。只把工程地基立起来。

量化交付 1	目录结构: 至少 8 个目录, 覆盖 include/src/examples/tests/scripts/docs。
量化交付 2	CMake: 顶层 CMakeLists.txt 能定义 C++20、warning flags、miniray_common、miniray_hello、status_test。
量化交付 3	代码: version.h、version.cc、hello_miniray.cc、status_test.cc 能编译。
量化交付 4	验证: cmake configure、build、ctest、hello demo 四条命令全部成功。
量化交付 5	文档: README 至少包含项目定位和完整构建/测试命令。
量化交付 6	版本控制: 完成一次语义清晰的 git commit。

2. 4 小时量化执行表

时间段	目标	具体动作	可量化完成标准
00:00-00:20	启动与环境确认	进入工作目录；确认 <code>git/cmake/compiler</code> 可用；创建 <code>miniray</code> 仓库。	<code>git init</code> 成功； <code>cmake --version</code> 可输出。
00:20-00:50	目录与忽略规则	创建 <code>include/src/examples/tests/scripts/docs</code> ；写 <code>.gitignore</code> 。	8+ 个目录存在； <code>build/</code> 被忽略。
00:50-01:30	CMake 骨架	写顶层 <code>CMakeLists</code> ；配置 <code>C++20</code> 、 <code>warning flags</code> 、 <code>library</code> 、 <code>demo</code> 、 <code>test</code> 。	<code>cmake -S . -B build</code> 通过。
01:30-02:10	最小 C++ 代码	实现 <code>version.h/version.cc</code> ；实现 <code>hello_miniray.cc</code> 。	<code>miniray_hello</code> 能链接 <code>miniray_common</code> 。
02:10-02:45	测试入口	实现 <code>status_test.cc</code> ； <code>enable_testing</code> ； <code>add_test</code> 。	<code>ctest</code> 能发现并运行 <code>status_test</code> 。
02:45-03:15	构建与修复	完整跑 <code>configure/build/ctest/demo</code> ；修复 <code>warning</code> 和路径错误。	4 条验收命令全部成功。
03:15-03:40	README 与复现命令	写 <code>README</code> ：定位、 <code>Build</code> 、 <code>Test</code> 、 <code>Run</code> 、目录说明。	别人照 <code>README</code> 能复现。
03:40-04:00	记录与提交	写 <code>Day1</code> 完成记录； <code>git status</code> ； <code>git add</code> ； <code>git commit</code> 。	<code>commit message</code> 清晰，工作区干净。

3. 目标目录结构

今天结束时应看到的文件树

```
miniray/
├── CMakeLists.txt
├── .gitignore
├── README.md
├── include/
│   ├── miniray/
│   │   ├── common/
│   │   └── version.h
│   └── src/
│       ├── common/
│       └── version.cc
├── examples/
│   ├── cpp/
│   └── hello_miniray.cc
├── tests/
│   ├── unit/
│   └── status_test.cc
├── scripts/
└── docs/
    ├── weekly-notes/
    └── ray-compare/
```

4. 逐文件 TODO

文件	优先级	内容要求	验收标准
CMakeLists.txt	必须	定义项目、C++20、warning flags; 添加 miniray_common、miniray_hello、status_test; 启用 ctest。	configure/build/ctest 全部通过。
include/miniray/common/version.h	必须	声明 miniray::Version()。	被 demo 和 test 同时 include。
src/common/version.cc	必须	实现 miniray::Version(), 返回版本字符串。	无 warning; 可链接。
examples/cpp/hello_miniray.cc	必须	打印 Hello from MiniRay 版本信息。	./build/miniray_hello 有可读输出。
tests/unit/status_test.cc	必须	用 assert 做 smoke test; 先不强依赖 GoogleTest。	ctest --test-dir build 成功。
README.md	必须	写项目定位、构建、测试、运行命令。	复制命令即可复现。
docs/weekly-notes/week01_day01.md	建议	记录今天遇到的问题、修复方式、明日待办。	不少于 8 行。
docs/ray-compare/week01.md	建议	只写工程层面对比: Ray 是多组件多进程工程; 今天不读源码。	100-200 字即可。

5. 可直接执行的命令清单

Day 1 命令总览

```
# 1) 创建项目
mkdir -p miniray && cd miniray
git init

# 2) 创建目录
mkdir -p include/miniray/common src/common examples/cpp tests/unit scripts docs/weekly-notes docs/ray-compare

# 3) 构建与测试
cmake -S . -B build
cmake --build build
ctest --test-dir build
./build/miniray_hello

# 4) 提交
git status
git add .
git commit -m "week1 day1: initialize miniray project skeleton"
```

6. 最小代码模板

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)

project(MiniRay
  VERSION 0.1.0
  LANGUAGES CXX
)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_compile_options(-Wall -Wextra -Wpedantic)

add_library(miniray_common
  src/common/version.cc
)

target_include_directories(miniray_common
  PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include
)

add_executable(miniray_hello
  examples/cpp/hello_miniray.cc
)

target_link_libraries(miniray_hello PRIVATE miniray_common)

enable_testing()

add_executable(status_test
  tests/unit/status_test.cc
)

target_link_libraries(status_test PRIVATE miniray_common)
add_test(NAME status_test COMMAND status_test)
```

version.h

```
// include/miniray/common/version.h
#pragma once

#include <string>

namespace miniray {

std::string Version();

} // namespace miniray
```

version.cc

```
// src/common/version.cc
#include "miniray/common/version.h"

namespace miniray {
```

```
std::string Version() {  
    return "MiniRay 0.1.0";  
}  
  
} // namespace miniray
```

hello_miniray.cc

```
// examples/cpp/hello_miniray.cc  
#include <iostream>  
  
#include "miniray/common/version.h"  
  
int main() {  
    std::cout << "Hello from " << miniray::Version() << std::endl;  
    return 0;  
}
```

status_test.cc

```
// tests/unit/status_test.cc
#include <cassert>
#include <string>

#include "miniray/common/version.h"

int main() {
    assert(!miniray::Version().empty());
    assert(miniray::Version().find("MiniRay") != std::string::npos);
    return 0;
}
```

7. Done Definition: 今天收工前必须打勾

验收清单

☐ git init 已完成	☐ 目录结构符合目标树
☐ .gitignore 已忽略 build/	☐ CMake 使用 C++20
☐ 已开启 -Wall/-Wextra/-Wpedantic	☐ miniray_common 可编译
☐ miniray_hello 可运行	☐ status_test 已加入 ctest
☐ cmake -S . -B build 通过	☐ cmake --build build 通过
☐ ctest --test-dir build 通过	☐ ./build/miniray_hello 输出版本信息
☐ README 包含 Build/Test/Run	☐ 记录了至少 3 个今天学到的点
☐ 记录了至少 1 个 blocker 或疑问	☐ 完成 git commit

8. 常见故障与处理

现象	可能原因	处理方式
找不到头文件	target_include_directories 没有给 miniray_common 设置 include 根目录。	确认 include 路径是 \${CMAKE_CURRENT_SOURCE_DIR}/include。
链接失败	可执行程序没有 link miniray_common。	target_link_libraries(miniray_hello PRIVATE miniray_common)。
ctest 找不到测试	忘记 enable_testing() 或 add_test()。	顶层 CMakeLists 增加 enable_testing 和 add_test。
build 目录污染 git	.gitignore 未生效或已经被 add。	确认 .gitignore 有 build/；必要时 git rm -r --cached build。
warning 太多	编译选项打开后暴露了未使用变量/头文件。	今天要修干净，不要带 warning 进入 Day2。

9. 今日复盘模板

复制到 `docs/weekly-notes/week01_day01.md`

```
# Week 01 Day 01 Review

## 今天完成了什么
- [ ]

## 四条验收命令结果
- cmake configure:
- build:
- ctest:
- hello demo:

## 今天遇到的问题
1.
2.
3.

## 我对 CMake target 的理解
- miniray_common:
- miniray_hello:
- status_test:

## 明天进入 Day2 前要确认
- [ ] build 目录可删除后重新构建
- [ ] README 命令可复现
- [ ] 没有 warning
```

10. 今天不要做

- 不要接 gRPC/Protobuf；那是 Week 13 的事情。
- 不要写 ThreadPool；那是 Week 3 的事情。
- 不要写 ObjectRef/ObjectStore；那是 Week 6-7 的事情。
- 不要深挖 Ray 源码；Week 1 只理解工程形态。
- 不要为了“漂亮架构”过早拆太多 target；今天先保证简单、可跑、可复现。

Day 1 的胜利标准不是“写了很多代码”，而是：从今天开始，MiniRay 每一行代码都有稳定的工程入口。